МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №3
по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: В.В. Рыбин
Группа: М8О-311Б-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы: загрузить изображения из файловой системы, создать из них текстуры Vulkan и с помощью текстурных координат в вершинах и сэмплеров наложить их на объекты сцены, сохранив освещение по модели Блинна-Фонга из лабораторной работы №2.
Сцена состоит из куба и двух конусов, которые освещаются направленным, точечными и прожекторными источниками света.
Программа написана на C++ с использованием Vulkan, GLFW, ImGui, Vulkan Memory Allocator (VMA) и vk-bootstrap.
Выполнено:

- базовое задание: текстура-альбедо на всех объектах вместо сплошного цвета; один динамический uniform-буфер со свойствами моделей и один набор дескрипторов с парой «изображение-сэмплер»; вершинные и индексные данные в видеопамяти (VRAM) через staging-буфер;
- дополнительное задание 1: несколько текстур, у каждой модели своя, выбор через выпадающий список ImGui; для каждого материала свой набор дескрипторов;
- дополнительное задание 2: для одного объекта нетривиальное сэмплирование (модуляция цвета и двойная выборка с разными координатами, зависящие от времени), включаемое флагом в push-константах;
- дополнительное задание 3: specular- и emissive-текстуры вместе с альбедо, для отсутствующих карт используется служебная текстура из одного чёрного пикселя.

Метод решения

1. Текстурные координаты

Вершина теперь содержит позицию, нормаль и текстурные координаты (u, v), атрибут цвета удалён. Начало координат текстуры находится в левом верхнем углу изображения.

• Куб. У каждой из шести граней четыре отдельные вершины (нормали у граней разные). Вершины идут в порядке «правый верх, левый верх, левый низ, правый низ» при взгляде снаружи, поэтому на всех гранях используется одна раскладка: (1,0), (0,0), (0,1), (1,1). Каждая грань показывает картинку целиком.

• Боковая поверхность конуса. Для вершины i из n на окружности основания u = i / n, v = 1. Для вершины конуса v = 0. Чтобы на стыке последнего и первого сегментов не было шва, кольцо основания содержит n + 1 вершин (последняя повторяет первую с u = 1). Вершина конуса продублирована для каждого сегмента с u = (i + 0,5) / n.

• Основание конуса. Используется плоская проекция круга на текстуру: u = 0,5 + 0,5·cos α, v = 0,5 + 0,5·sin α, центр основания получает (0,5; 0,5).

2. Создание текстуры в Vulkan

Изображение читается из файла библиотекой stb_image и приводится к формату RGBA8 (4 канала). Далее создаются:

• VkImage: двумерное, один MIP-уровень, один слой, расположение в памяти VK_IMAGE_TILING_OPTIMAL, использование TRANSFER_DST | SAMPLED. Память выделяется через VMA на стороне GPU;

• VkImageView с тем же форматом и цветовым аспектом;

• VkSampler: фильтрация LINEAR при увеличении и уменьшении, адресация REPEAT по осям U и V, один MIP-уровень, анизотропная фильтрация выключена.

Область памяти VRAM недоступна для записи с процессора, поэтому пиксели сначала копируются в staging-буфер: буфер размером 64 МБ в памяти, видимой процессору, с использованием TRANSFER_SRC. Он работает как линейный аллокатор: смещение сдвигается на размер каждой новой порции данных, а смещения выравниваются по 16 байт. Команды копирования записываются в одноразовый командный буфер из собственного командного пула и отправляются в очередь один раз. Процессор ждёт их выполнения на VkFence (vkWaitForFences с UINT64_MAX).

Перед копированием и после него расположение изображения меняется барьерами VkImageMemoryBarrier:

1. UNDEFINED → TRANSFER_DST_OPTIMAL: стадия TOP_OF_PIPE → TRANSFER, доступ на запись при копировании;
2. vkCmdCopyBufferToImage из staging-буфера в изображение;
3. TRANSFER_DST_OPTIMAL → SHADER_READ_ONLY_OPTIMAL: стадия TRANSFER → FRAGMENT_SHADER, доступ на чтение из шейдера.

Формат выбирается по смыслу карты. Альбедо и emissive хранят цвет, поэтому для них используется VK_FORMAT_R8G8B8A8_SRGB: при выборке значение автоматически переводится в линейное пространство по формуле c_лин = ((c + 0,055) / 1,055)^2,4. Освещение считается именно в линейном пространстве, а итоговый цвет кодируется обратно в sRGB при записи в кадровый буфер. Specular-карта хранит маску, а не цвет, поэтому для неё используется VK_FORMAT_R8G8B8A8_UNORM (без преобразования).

3. Вершинные и индексные данные в VRAM

Вершинные и индексные буферы теперь создаются с использованием VERTEX_BUFFER_BIT | TRANSFER_DST_BIT (и INDEX_BUFFER_BIT | TRANSFER_DST_BIT) и выделяются в памяти GPU без флагов доступа с хоста. Данные попадают в них тем же способом: копирование в staging-буфер и команда vkCmdCopyBuffer. Так же переведён буфер линий для визуализации конусов прожекторов.

4. Динамический uniform-буфер

Вместо отдельного uniform-буфера и набора дескрипторов на каждый объект используется один буфер, в котором лежит массив структур ModelUniforms (матрица модели, shininess, цвет блика). Дескриптор имеет тип VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC, а смещение к нужной структуре задаётся не при записи дескриптора, а в команде привязки.

Элементы в таком буфере нельзя считать упакованными плотно: смещение между ними обязано быть кратно значению minUniformBufferOffsetAlignment из лимитов физического устройства. Поэтому шаг массива считается так:

stride = (sizeof(ModelUniforms) + a − 1) & ~(a − 1), где a = minUniformBufferOffsetAlignment.

Например, при a = 256 структура в 96 байт занимает в массиве 256 байт. Размер буфера равен stride × MAX_MODELS (MAX_MODELS = 1024). В VkDescriptorBufferInfo указывается range = sizeof(ModelUniforms), то есть размер одной структуры, а не всего буфера. Для объекта с индексом i данные записываются по адресу memory + i·stride, а при отрисовке в vkCmdBindDescriptorSets передаётся динамическое смещение offset = i·stride.

5. Наборы дескрипторов

У каждого материала свой набор дескрипторов (set = 1), построенный по одному и тому же layout.

Набор set = 0 общий для сцены: uniform-буфер с камерой и параметрами света и два storage-буфера со списками точечных и прожекторных источников. Пул дескрипторов выделяет все использованные типы: UNIFORM_BUFFER, UNIFORM_BUFFER_DYNAMIC, STORAGE_BUFFER и COMBINED_IMAGE_SAMPLER, с запасом по числу наборов. При отрисовке объекта привязываются наборы {сцена, набор материала объекта} с одним динамическим смещением, поэтому смена текстуры объекта сводится к смене индекса материала.

6. Освещение с текстурами

В шейдере вместо сплошного цвета используются значения карт: T_a (альбедо), T_s (specular), T_e (emissive) в точке uv. Цвет пикселя по модели Блинна-Фонга.

Specular-карта T_s выступает маской: блеск виден только там, где она светлая. Emissive-слагаемое T_e прибавляется после расчёта освещения и от источников света не зависит, поэтому объект светится в темноте.

Если у материала нет specular- или emissive-карты, вместо неё привязывается служебная текстура 1×1 с чёрным пикселем. Тогда T_s = 0 (нет бликов) и T_e = 0 (нет свечения), а шейдер остаётся без ветвлений.

7. Нетривиальное сэмплирование и push-константы

Для одного объекта (второго конуса) включается особый режим выборки альбедо. Управление им вынесено в push-константы, а не в структуру ModelUniforms, как требует задание. Структура push-констант занимает 16 байт: uint32_t flags (бит FLAG_FANCY), float time, два байта-выравнивателя в виде float _pad[2]. Диапазон VkPushConstantRange объявлен для фрагментного шейдера и задан в layout основного пайплайна. Вместо bool используется uint, потому что размеры bool в C++ (1 байт) и в GLSL (4 байта) различаются. Значения передаются командой vkCmdPushConstants перед каждым вызовом отрисовки: флаг установлен только у выбранного объекта.

При включённом флаге текстура выбирается дважды с разными координатами, затем результаты смешиваются и окрашиваются, всё зависит от времени t:

• первая выборка со сдвигом: $uv_1 = uv + (0{,}1 \cdot t;\ 0)$, то есть бегущая текстура;

• вторая выборка с поворотом вокруг центра и масштабированием: $uv_2 = s \cdot R(0{,}5 \cdot t) \cdot (uv - 0{,}5) + 0{,}5$;
• смешивание: $c = \mathrm{mix}(T(uv_1), T(uv_2), w)$, $w = 0{,}5 + 0{,}5 \cdot \sin t$;
• модуляция цвета: $c' = c \cdot (0{,}6 + 0{,}4 \cdot \sin(t + \varphi_k))$ покомпонентно, где $\varphi = 0$; 2,1; 4,2 для R, G, B.
В UI это включается чекбоксом, а время берётся из часов GLFW, поэтому эффект анимируется независимо от остальных анимаций.
8. Интерфейс
В окне управления ImGui добавлен раздел выбора текстур: для каждого объекта выпадающий список материалов, а также настройки режима нетривиального сэмплирования. Остальные органы управления из лабораторной работы №2 (камера, источники света, анимация) сохранены

Результаты

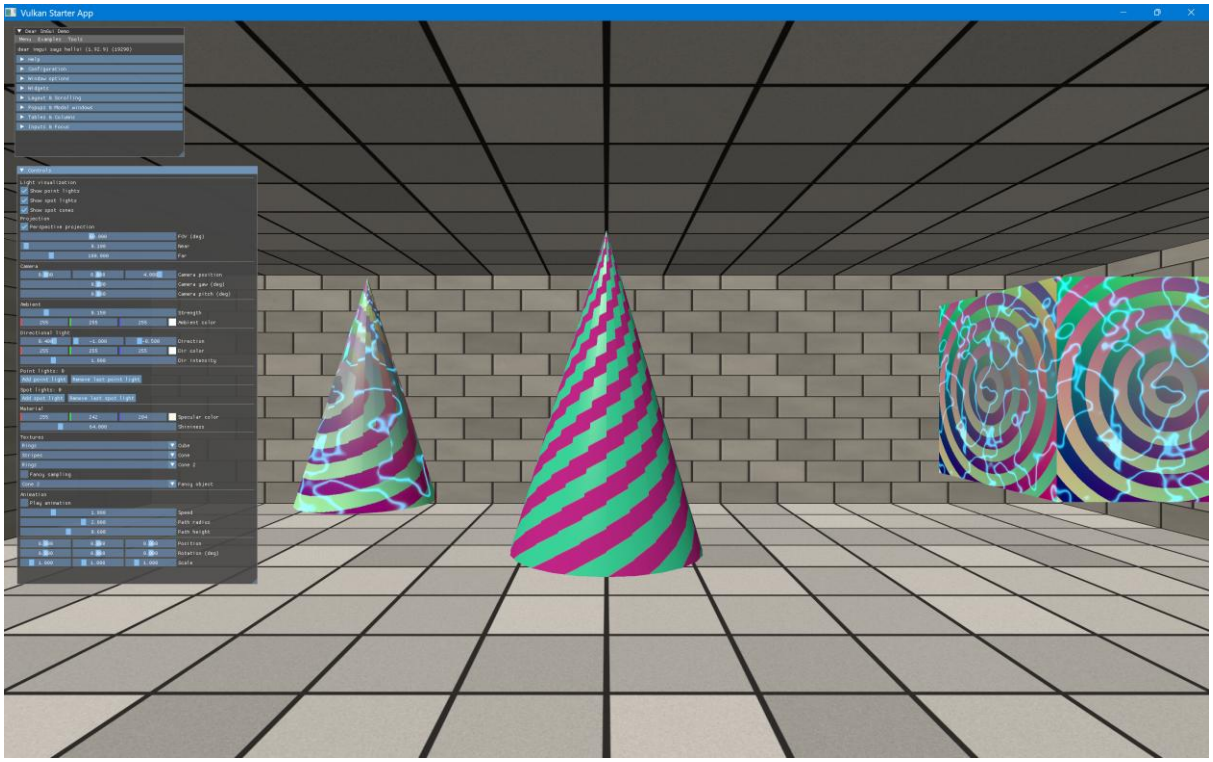


Рисунок 1. Куб и два конуса с разными текстурами в качестве альбедо; освещение Блинна-Фонга работает поверх текстур, у каждого объекта текстуру можно менять на лету.

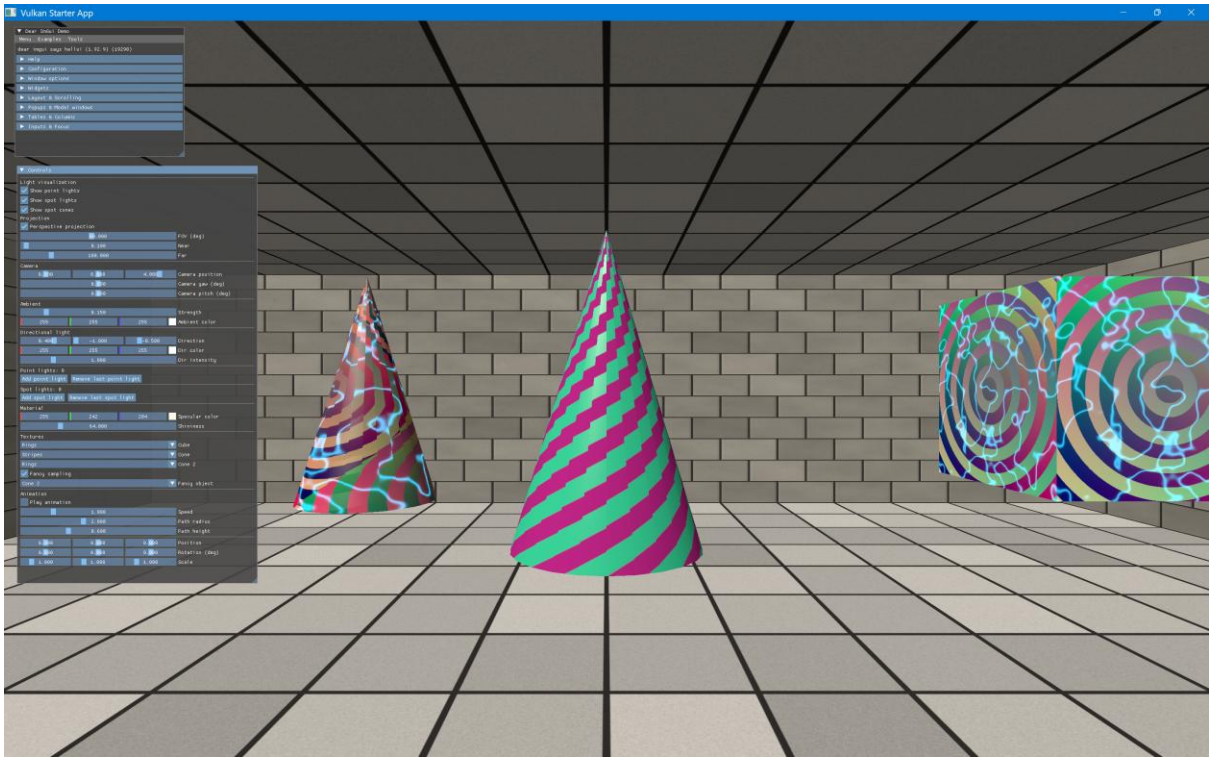


Рисунок 2. Нетривиальное сэмплирование на втором конусе: текстура смещается, смешивается со своей повёрнутой копией и меняет оттенок со временем. У остальных объектов выборка обычная.

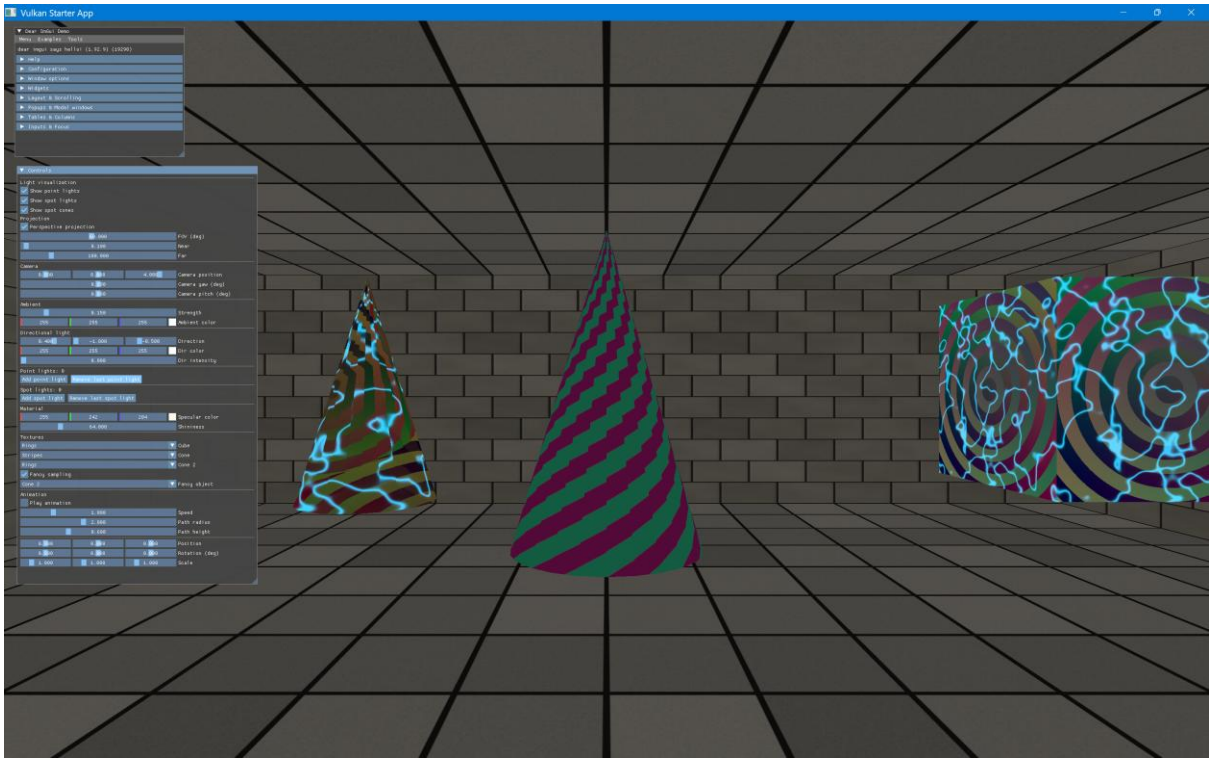


Рисунок 3. Specular- и emissive-карты: блик есть только на светлых участках specular-карты, emissive-участки светятся в тёмной сцене. У

материала без этих карт нет ни бликов, ни свечения, так как вместо них используется чёрный пиксель.

Выводы

В ходе работы я научился загружать изображения из файла и превращать их в текстуры Vulkan: создавать VkImage, VkImageView и VkSampler, копировать пиксели в видеопамять через staging-буфер и одноразовый командный буфер, а также управлять расположением изображения в памяти с помощью барьеров.
Я разобрался, зачем нужны текстурные координаты и как процедурно получать их для куба и конуса, в том числе как избежать шва на боковой поверхности. Я понял различие форматов sRGB и UNORM: цветовые карты хранятся в sRGB, а маски (specular) в UNORM, иначе освещение считается в неверном цветовом пространстве.
Я научился использовать динамический uniform-буфер: один буфер и один набор дескрипторов на множество объектов, где смещение задаётся при привязке, а шаг массива должен учитывать minUniformBufferOffsetAlignment. Я перенёс вершинные и индексные буферы в память GPU, а также применил push-константы для передачи небольших данных (флаг и время) прямо в командном буфере без отдельных ресурсов, узнав об ограничениях на их размер и выравнивание.
Наконец, я реализовал specular- и emissive-карты и увидел, как маска блеска и самосвечение управляют внешним видом материала независимо от модели освещения.